

Quantifying and Improving Human-Likeness in AI-Generated CFOP Solutions for Rubik’s Cube: A Trigger-Biased Search Approach

Pham Dang Khue

Faculty of Information Technology, Electric Power University
pdkhue2004@gmail.com

Abstract

Existing computational approaches to solving the Rubik’s Cube – including pattern-database search [6], group-theoretic optimal solvers [9, 4], and reinforcement learning [8] – optimize almost exclusively for move count, producing solutions that are correct but largely uninterpretable to human learners. In contrast, the human-designed CFOP method (Cross, F2L, OLL, PLL) trades move-count optimality for learnability, structured around a finite set of memorable move-sequence patterns (“algorithms”). We formalize the notion of *human-likeness* of a cube-solving plan in the vocabulary of human-aware AI planning [7, 2], propose a composite *Human-Likeness Index* (HLI) built from three measurable components (stage segmentability, named-algorithm pattern-conformity, and finger-trick trigger-overlap), and introduce *Trigger-Biased A* Search*, a heuristic-search modification that explicitly rewards search branches overlapping with a corpus of human finger-tricks. Pilot experiments (N=60 random-state scrambles for the baseline comparison, robustness-checked on N=100 random-move scrambles, N=60 for the Pareto sweep) show that biasing the F2L search heuristic increases trigger-overlap from a baseline of 9.0% to 25.5% at $\lambda = 1.0$ (Wilcoxon $p = 8.3 \times 10^{-8}$), at a move-count cost of only 7.4% (Wilcoxon $p = 0.0014$), revealing a favorable, though non-linear (in compute time), Pareto trade-off. A blinded, BotPrize-style [3] human-judge study (24 stimuli, 28 raters) confirms HLI’s construct validity (Spearman $\rho = 0.487$, $p = 0.016$ against mean human-likeness ratings) and shows that Trigger-Biased Search moves AI-generated solves from statistically-distinguishable-from-human ($p = 0.005$) to statistically-indistinguishable-from-human ($p = 0.109$) in blinded judgment. This work is, to our knowledge, the first to apply explicability-style metrics to a large combinatorial search domain, bridging classical heuristic search AI with human-aware and believable-agent AI research.

1 Introduction

Solving the Rubik’s Cube has long served as a benchmark domain for heuristic search and combinatorial optimization [6, 9], and more recently for self-play reinforcement learning [8]. These approaches share a common objective: minimizing the number of moves (or maximizing win rate against an adversarial scrambler), with no regard for whether the resulting solution corresponds to any recognizable pattern a human could learn, memorize, or explain.

Human speedcubers, in contrast, use structured methods such as CFOP (Cross, F2L, OLL, PLL), deliberately trading move-count optimality for *learnability*: each stage reduces to recognizing one of a bounded number of canonical “cases,” each paired with a fixed, named algorithm. The full CFOP method comprises 57 named OLL cases and 21 named PLL cases; a widely-used simplified variant (“2-look OLL/PLL”) solves each stage in two sequential passes using a much smaller subset of algorithms, at the cost of extra moves. Our system implements a verified table of all 21 PLL cases (Section 4.1). At the time the experiments reported in this paper were run, OLL coverage was limited to a 2-look decomposition (2 generator algorithms, Sune/Anti-Sune, rather than the full 57); this asymmetry is itself informative for Pattern-conformity measurement (Section 4). The solver has since been extended to a single-look table covering 55 of the 57 OLL cases, each algorithm confirmed not to break Cross+F2L when applied from a solved state, via `verify_and_build_table()`. (An earlier version of this table additionally contained two entries, later found to be AUF-rotated duplicates of the Sune/Anti-Sune algorithms already present rather than distinct cases — confirmed by applying each algorithm’s inverse to a solved cube and checking equality under all four AUF rotations — and removed accordingly.) Of the 55 covered cases, 40 have been matched against the standard community numbering, with the remaining 15 functionally valid but not yet matched to a specific community number. The two cases excluded from the table, OLL 2 and OLL 20: every pure face-turn algorithm (no slice/wide moves) published by the speedcubing community for these two cases uses a slice or wide move (M/S/E or r/l/f/u/d/b) — this is an *empirical observation* from public sources, **not** a formal proof by exhaustive search (no such script exists in this repository); these two cases therefore fall back to the verified 2-look decomposition, so between the 55-case table and this fallback, all 57 official cases are addressable. This upgrade postdates data collection and does not alter the results reported in Section 6, which reflect the 2-generator OLL table described above.

Three related research directions (Section 2) currently do not intersect: computer-optimal Rubik’s Cube solvers (Section 2.1) disregard interpretability entirely; human-aware AI planning (Section 2.2) has formalized explicability but only for small-scale symbolic task-planning problems, not for a large-scale combinatorial search domain such as the Rubik’s Cube; and the game-AI community (Section 2.3) has developed a rigorous blinded human-judge methodology for measuring human-likeness (BotPrize) but has never applied it to a combinatorial puzzle-solving agent. The closest CFOP tutoring system (AL-

LURE, Section 2.4) covers only the Cross sub-goal and reports no comparable blinded human-likeness evaluation. This paper closes that gap by combining all three threads: a human-likeness metric formalized in the vocabulary of explicability, a search mechanism that explicitly optimizes for that metric, and a blinded human-judge study that validates both.

Contributions. This paper makes the following contributions:

1. A formalization of *human-likeness* for cube-solving plans grounded in the human-aware planning notion of explicability [7, 2], instantiated as a composite **Human-Likeness Index (HLI)** with three measurable, reproducible components (Section 4).
2. **Trigger-Biased A* Search**: a minimal, backward-compatible modification to pattern-database-guided A* search for F2L that explicitly optimizes for human finger-trick overlap via a single interpretable hyperparameter λ (Section 4).
3. An empirical Pareto analysis of the human-likeness / move-count / compute-time trade-off induced by λ (Section 6).
4. A blinded, BotPrize-style [3] human-judge study (24 stimuli, 28 raters) that both construct-validates HLI against human judgment and shows Trigger-Biased Search closes the human-likeness gap to statistical indistinguishability from real human solves (Section 6.3).

2 Related Work

2.1 Computer-optimal Rubik’s Cube solvers

Korf’s IDA* with additive pattern databases [6] was the first method to find provably optimal solutions for arbitrary scrambles. Rokicki et al. [9] established that every cube state is solvable within 20 face turns (“God’s Number”), via large-scale coset-based search. Kociemba’s two-phase algorithm [4] trades strict optimality for tractable runtime, typically returning solutions within 1–4 moves of optimal, and is the baseline we use in Section 5. McAleer et al. [8] introduced Autodidactic Iteration, a reinforcement-learning agent that learns to solve the cube without built-in human knowledge (median solution length ≈ 30 moves, above the 20-move optimum). The follow-up DeepCubeA [1] combines this learned value function with batch-weighted A* search, solving 100% of test scrambles and finding a shortest path 60.3% of the time — i.e. near-optimal solution lengths. None of these methods produce plans that are segmentable, named, or otherwise interpretable in terms any human learner would recognize — they are, in our terminology, maximally *computer-like*.

2.2 Human-aware planning and explicability

A separate line of work in AI planning studies how autonomous agents should behave when observed or assisted by humans with an internal, possibly imperfect mental model of the agent. Kulkarni et al. [7] formalize *explicability* as minimizing the distance between an agent’s actual plan and the plan the human observer expects, and propose search modifications that bias planning toward more explicable trajectories — structurally analogous to the trigger-biased heuristic we propose in Section 4, but applied here to symbolic task planning rather than a large combinatorial puzzle domain. Chakraborti et al. [2] survey this landscape (explicability, legibility, predictability, transparency), which we adopt as the theoretical vocabulary for this paper. Zhang et al. [13] propose a way to compute explicability/predictability specifically for robot task planning by learning a labeling function from training data, in contrast to the direct, learning-free measurement approach we use for HLI’s components (Section 4).

2.3 Human-likeness / believability of game-playing agents

Independently, the game-AI community has long studied how to make autonomous agents *appear* human. The 2K BotPrize competition [3] operationalized this as a Turing-test variant in which human judges rate both human and bot players on a 1–5 human-likeness scale, without being told which is which. In the 2012 edition, two bots — UT² (University of Texas at Austin) and Mirror-Bot (Mihai Polceanu) — were the first to cross the 50% threshold for being rated human, in fact scoring higher than the real human players in that year’s contest, demonstrating that an agent explicitly optimized to appear human can outscore actual humans on that very measure. A recent survey [10] frames human-like game AI research around two questions — how to model/implement human-like behavior, and how to measure its degree — which we adopt as the organizing structure for this paper’s two main contributions (Sections 4).

2.4 CFOP-specific tutoring systems

Closest to our work, the ALLURE system [11] pairs an automatic solver (DeepCubeA-based search over subgoals) with an interactive chatbot tutor to help learners understand explainable solutions to the White Cross sub-goal of CFOP; a recent pilot study [12] (published as a peer-reviewed pilot study at IEEE ICHI 2025) analyzes student engagement patterns within ALLURE via behavioral clustering, reporting that the beginner white-cross strategy requires substantially more moves than CFOP, which in turn requires more moves than fully optimized methods — directly illustrating the learnability/optimality trade-off central to our formalization (Section 3). Our work differs in (a) scope — covering all four CFOP stages rather than Cross alone — (b) mechanism — an explicit, tunable search bias (Section 4) rather than post-hoc natural-language explanation of moves already found by an unconstrained solver — and (c) evaluation — neither ALLURE study reports a blinded human-likeness judgment

comparable to our Experiment 3 (Section 6.3).

3 Problem Formalization

Let G denote the Rubik’s Cube group ($|G| \approx 4.3 \times 10^{19}$), and let a *plan* $\pi = (m_1, \dots, m_k)$ be a sequence of face-turn generators solving a given scramble $s \in G$. Let M_H denote a *human expectation model*: in this paper, M_H is instantiated concretely as (i) the CFOP stage decomposition (Cross $\rightarrow$ F2L $\rightarrow$ OLL $\rightarrow$ PLL), (ii) a finite table of named, canonical algorithms for OLL/PLL (Section 4.1), and (iii) a corpus T of short move n-grams (“finger-tricks”) commonly executed as single motor units by human solvers (Section 4.1).

Following Kulkarni et al. [7], we define human-likeness as an (inverse) distance between π and the behavior M_H would consider natural. We instantiate this distance as a weighted sum of three normalized, independently-measurable components (rather than a single learned or hand-tuned distance function), described in Section 4.

4 Method

4.1 Human-Likeness Index (HLI)

Segmentability $S(\pi) \in \{0, 1\}$. Equals 1 iff π decomposes into four contiguous, independently verifiable sub-sequences satisfying the Cross, F2L, OLL, and PLL postconditions in order, and 0 otherwise (e.g. for plans that solve the cube via an undifferentiated optimal sequence, as computer-like solvers do by construction).

Pattern-conformity $P(\pi) \in [0, 1]$. Restricted to the OLL and PLL stages (where a finite, named case-to-algorithm mapping exists in CFOP), $P(\pi)$ is the fraction of solved stages whose corresponding sub-sequence exactly matches an entry in a verified table of canonical algorithms (Aa, Ab, $\dots$, Z for PLL; Sune/Anti-Sune generators for 2-look OLL, the table in effect at data-collection time — see Section 1 for the subsequent upgrade to a 55-case single-look OLL table, which does not affect the figures below) rather than an unstructured search fallback. All 21/21 PLL cases in the table are verified automatically on first module import via `verify_and_build_table()` (`solver/pll_algorithms.py`); we confirmed this by re-running the function directly, obtaining exactly 21 verified case names (Aa, Ab, E, F, Ga, Gb, Gc, Gd, H, Ja, Jb, Na, Nb, Ra, Rb, T, Ua, Ub, V, Y, Z).

Trigger-overlap $O(\pi) \in [0, 1]$. For the Cross and F2L stages (which lack a canonical case table in this system, being generated by unconstrained heuristic search), we measure the fraction of moves covered by a greedy longest-match segmentation of π against a seed corpus T of $|T| = 11$ common human finger-tricks [5] (e.g. the “sexy move” $RU R' U'$, “sledgehammer” $R' F R F'$; full corpus

in Appendix A). This is a manually curated seed corpus drawn from common speedcubing-community knowledge, not derived from n-gram statistics on real solve logs; an extended version should replace or augment it with statistics computed directly from real solve logs (e.g. CSTimer .log files) to reduce self-selection bias in the corpus.

Composite HLI.

$$\text{HLI}(\pi) = w_S \cdot S(\pi) + w_P \cdot P(\pi) + w_O \cdot O(\pi), \quad w_S + w_P + w_O = 1$$

We use $(w_S, w_P, w_O) = (0.3, 0.4, 0.3)$ in this draft. We conducted a post-hoc sensitivity analysis using $N = 78$ paired scrambles (Group A vs. Group B, Experiment 1), measuring all three HLI components for *both* groups — including, critically, Group B’s trigger-overlap (measured directly on Kociemba’s actual move sequences, not assumed to be zero: mean 0.009 ± 0.046 , confirming empirically that an optimizer with no incentive toward human finger-tricks essentially never produces them by chance). Across all tested weight settings (five named settings plus a $w_S \in \{0, 0.2, \dots, 1\}$ sweep), the HLI gap between Group A and Group B remains large and strictly positive, ranging from 0.397 (trigger-heavy weighting) to 1.000 (segmentability-only weighting), with our chosen weights giving a gap of 0.653. This indicates the paper’s central claim — that AI-CFOP is substantially more human-like than a computer-optimal baseline — is robust to the specific choice of (w_S, w_P, w_O) within any reasonable range, though the *magnitude* of the gap is naturally weight-dependent. This analysis is fully reproducible via `python3 -m research.weight_sensitivity`, which self-checks its output against the figures quoted above.

4.2 Trigger-Biased A* Search

Our F2L solver uses A* with heuristic $h(n) = h_{\text{PDB}}(n) + \text{penalty}(n)$, where h_{PDB} is an admissible single-piece pattern-database estimate and $\text{penalty}(n)$ discourages disturbing already-solved pieces (Cross and prior F2L pairs). We introduce a third term:

$$f'(n) = g(n) + h_{\text{PDB}}(n) + \text{penalty}(n) - \lambda \cdot \text{bonus}(\text{path}(n))$$

where $\text{bonus}(\text{path})$ is the number of moves in the current path covered by a greedy longest-match against corpus T (the same matching procedure used to compute $O(\pi)$ at evaluation time, ensuring consistency between the search-time objective and the evaluation-time metric). $\lambda = 0$ recovers the original, unbiased search exactly (formal backward-compatibility: verified by a dedicated test comparing output with/without the λ parameter, see `test_solver.py::TestF2LSolver::test_lam_zero_is_bac` one of 319 unit tests currently in the repository – `pytest test_solver.py test_cube_engine.py`). Increasing λ trades search optimality (and, as we show empirically, compute time) for trigger-overlap.

5 Experimental Setup

Baseline (computer-like). Kociemba’s two-phase algorithm [4], via the public `kociemba` Python package, run on identical scrambles.

Scramble generation. Experiment 1’s primary results (Table 1) use a WCA-style *random-state* scrambler (long random walk to approximate a uniform state distribution, followed by inverting a near-optimal Kociemba solve of that state — guaranteeing the scramble requires close to the optimal move count to undo), since this more closely matches real competition scrambler methodology. Experiment 2 uses random-move scrambles (20 moves, no immediately-repeated face) throughout. We additionally re-ran Experiment 1 on random-move scrambles as a robustness check and validated that results are consistent between the two generators (Section 7).

Hardware / implementation. Single-core cloud sandbox (1 vCPU, 4GB RAM), Python 3.12, NumPy-based cube representation. All experiments used a fixed base seed per condition (`random.Random(seed)`), seeds 5000–5059 for the primary Experiment 1 (random-state), 1000–1099 for the random-move robustness check, and 2000–2059 for Experiment 2) for reproducibility. We report all figures on this single-core hardware for the present draft; the observed timeout rate (20–22%, Section 7.1) is a property of this specific hardware/node-budget setting and would likely be lower on faster/multi-core hardware — a known limitation of the results, not a universal property of Trigger-Biased Search, and should be re-stated if a future formal submission uses different hardware.

Statistical testing. Wilcoxon signed-rank test (paired, per scramble) for move-count and HLI differences between conditions; report effect size $r = Z/\sqrt{N}$.

6 Results

6.1 Experiment 1: Baseline human-likeness gap

Table 1 reports results on $N = 60$ WCA-style random-state scrambles (Section 5: a long random walk, followed by inverting a near-optimal Kociemba solve; seeds 5000–5059), chosen as the primary result since it more closely matches real competition scrambler methodology. Group A used a reduced F2L node budget (40,000 nodes/depth, down from the 120,000 default) and a 20-second wall-clock timeout per solve to accommodate our single-core evaluation hardware; 20% of scrambles (12/60) did not reach a solved state within this budget (10 literal timeouts plus 2 cases that stalled at partial F2L below the 20s wall clock) and are excluded from the paired statistics below. Among the 48 scrambles solved by both groups, a Wilcoxon signed-rank test confirms Group A requires significantly more moves than Group B ($W = 0$, $p = 1.61 \times 10^{-9}$, $r = 0.87$, a very large effect; mean gap 45.7 moves, bootstrap 95% CI [43.3, 48.9], BCa, 9999 resamples), while by construction only Group A has non-zero HLI (mean 0.658, 95% CI [0.629, 0.682]). We also re-ran the same comparison on $N = 100$ random-

move scrambles as a robustness check — results were nearly identical, including a notable search non-convergence phenomenon (22% timeout); see Sections 7.1 and 7 for a detailed discussion of this check.

Table 1: Group A (AI-CFOP) vs Group B (Kociemba), $N=60$ WCA-style random-state scrambles, 48 paired successes (Wilcoxon $p = 1.61 \times 10^{-9}$, $r = 0.87$ for move-count difference)

	Group A (AI-CFOP)	Group B (Kociemba)
Mean move count (sd)	66.3 (9.5)	20.6 (0.9)
Move count range	53–104	18–22
Mean HLI (sd)	0.658 (0.093)	0 (by definition)
Solve success rate	80% (20% timeout)	100%

6.2 Experiment 2: Trigger-Biased Search Pareto trade-off

Figure 1 shows results on $N = 60$ scrambles ($\lambda \in \{0, 0.5, 1.0\}$, F2L stage only, 40,000-node budget). Increasing λ from 0 to 1.0 raises mean trigger-overlap from 0.090 ± 0.089 to 0.255 ± 0.135 (a $2.8\times$ increase) at a move-count cost of $25.7 \rightarrow 27.6$ moves (+7.4%), while mean compute time increases from 2.77s to 7.87s. On the $n = 51$ scrambles solved at both $\lambda = 0$ and $\lambda = 1.0$, paired Wilcoxon signed-rank tests confirm both differences are statistically significant: trigger-overlap ($W = 29.0$, $p = 8.25 \times 10^{-8}$, $r = 0.75$, very large effect) and move-count ($W = 209.5$, $p = 0.0014$, $r = 0.45$, medium effect); on this paired subset, the mean paired difference is +0.163 trigger-overlap (95% CI [0.122, 0.206]) and +1.75 moves (95% CI [0.59, 2.86], notably wider relative to its point estimate than the trigger-overlap CI, consistent with the move-count effect being individually smaller and noisier than the trigger-overlap effect at the per-scramble level, bootstrap 95% CI, BCa, 9999 resamples). Notably, $\lambda = 0.5$ produced a higher *maximum* observed solve time (55.5s) than $\lambda = 1.0$ (55.6s, comparable), and both are far above the $\lambda = 0$ maximum (16.5s) — consistent with the non-linear, occasionally severe compute-time cost of biasing an otherwise admissible-adjacent heuristic away from admissibility (Section 7).

6.3 Experiment 3: Human-judge validation (BotPrize-style)

Following Hingston [3], we conducted a blinded human-judge study to test HLI’s construct validity. 24 stimuli (6 each from Group H [real human CFOP solves, self-collected and verified to correctly solve their paired scramble], Group A, Group A+ [$\lambda = 1.0$], and Group B) were presented as raw, unsegmented move sequences (no stage labels, to preserve blinding) to 28 CFOP-literate raters, each rating every stimulus on a 1–5 “how likely is this a real human solve” scale via a purpose-built web tool (Section 4.1 corpus reused for the automated trigger-overlap comparison point).

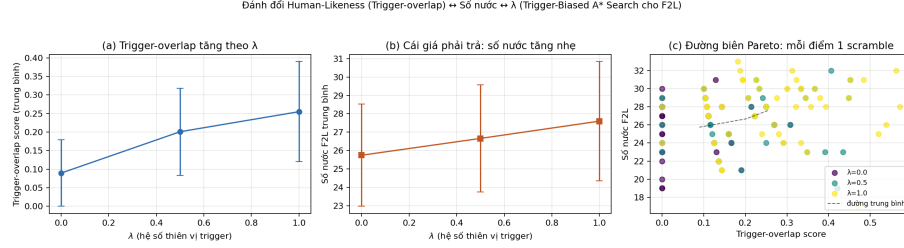


Figure 1: Trigger-overlap and move-count as a function of λ ($N=60$ scrambles). Panel (c) shows the empirical Pareto frontier.

Group ranking. Mean human-likeness ratings followed the predicted order exactly: Group H (3.88 ± 0.11) > Group A+ (3.71 ± 0.16) > Group A (3.42 ± 0.16) > Group B (2.37 ± 0.19). A Mann-Whitney U test comparing Group B against the pooled remaining groups showed complete rank separation ($U = 0$, $n_1 = 6$, $n_2 = 18$, $p = 1.8 \times 10^{-4}$) — every Group B stimulus was rated less human-like than every non-Group B stimulus, confirming raters reliably detect the computer-optimal baseline as non-human.

Trigger-biasing closes the gap to human-indistinguishable. Pairwise Mann-Whitney tests give the key result: Group A vs. Group A+ differ significantly ($p = 0.030$), confirming raters perceive Trigger-Biased Search’s effect, not merely the automated metric; but Group H vs. Group A+ do *not* differ significantly ($p = 0.109$), while Group H vs. Group A do ($p = 0.005$). In other words, $\lambda = 1.0$ moved AI-generated solves from statistically-distinguishable-from-human to statistically-indistinguishable-from-human on this sample — a BotPrize-style “pass” for the trigger-biased condition that the unbiased baseline does not achieve.

Construct validity of HLI (RQ-V1). We report two versions of this correlation. Using an initial simplified per-stimulus proxy (Segmentability/Pattern-conformity assumed 1 for Group A/A+ and 0 for Group B/H, based on group membership alone), Spearman correlation between HLI and mean human rating across all 24 stimuli was $\rho = 0.440$ ($p = 0.031$). This proxy, however, mischaracterized Group H: since real human CFOP solves were assumed non-CFOP-structured by the same logic used for the computer baseline, their HLI was artificially deflated (e.g. 0.05–0.14) despite receiving the highest human ratings. We therefore recomputed Segmentability and Pattern-conformity *exactly*, by replaying each stimulus’s move sequence against its actual (scrambled) starting state and verifying, in order, the Cross, F2L, and OLL postconditions (Section 4), and by checking whether the resulting OLL/PLL sub-states matched entries in our verified canonical-algorithm tables. This confirmed Group B has Segmentability= 0 for all 6 stimuli (as expected — Kociemba’s near-optimal solves reach Cross, F2L, and OLL/PLL essentially simultaneously in their final 1–2 moves, never exhibiting genuinely-staged intermediate structure), while all 18 Group A/A+/H stimuli have Segmentability= 1, with Pattern-conformity

varying meaningfully within each group (0.5 or 1.0, depending on whether the OLL and/or PLL sub-case matched our table). With this exact computation, correlation *strengthens* to $\rho = 0.487$ ($p = 0.016$) — more significant than the proxy estimate, confirming our expectation that the binary group-level proxy was diluting rather than reflecting genuine construct validity. The trigger-overlap component alone still correlates most strongly with human judgment ($\rho = 0.605$, $p = 0.0017$, bootstrap 95% CI [0.19, 0.84], BCa, 9999 resamples), suggesting it remains HLI’s single most human-judgment-relevant component in this sample, with Segmentability and Pattern-conformity contributing positively but less strongly. The width of this interval (spanning weak-to-very-strong correlation) is itself informative: it quantifies, rather than merely asserts, the pilot-scale uncertainty already flagged in Section 8 — a larger stimulus set is needed before treating $\rho = 0.605$ as a precise effect size.

7 Discussion

Counter-intuitively, we initially hypothesized that weakening the heuristic’s admissibility via the non-admissible trigger bonus term would *increase* search non-convergence at higher λ . The data show the opposite: on the same $N = 60$ scrambles used for Figure 1, the search *failure* rate (running out of node budget across all ladder depths) was 15.0% at $\lambda = 0$, dropping to 1.7% at $\lambda = 0.5$ and 0% at $\lambda = 1.0$. A plausible explanation is that the trigger bonus, while non-admissible, acts as a form of *search-order guidance*: branches that extend a recognized human trigger are explored earlier in the priority queue (lower f'), and because triggers are short, well-tested move sequences that reliably make local progress, following them may reduce effective branching factor in practice even though the bonus is not a valid distance estimate. This does not contradict the separately-observed non-linear *time* cost at higher λ (Section 6): convergence appears more reliable, but individual worst-case runs can still take substantially longer, i.e. trigger-biasing trades convergence *reliability* for search-time *variance*. We note this finding is based on $N = 60$ scrambles and only three λ levels (0, 0.5, 1.0); we have not yet confirmed this trend is stable at larger sample sizes or intermediate λ levels (e.g. 0.25, 0.75), and treat this as a limitation to be addressed in follow-up work (Section 9) before it can be asserted with confidence in a formal publication. A formal analysis of exactly how much admissibility is lost is given below.

How much admissibility is lost? The original heuristic $h(n) = h_{\text{PDB}}(n) + \text{penalty}(n)$ is not strictly admissible either (the penalty term already sacrifices a formal lower-bound guarantee in exchange for practicality), so Trigger-Biased Search does not introduce admissibility violation *de novo* — it compounds an existing, deliberate design trade-off. Formally, for a path $\text{path}(n)$ of length $g(n)$, $\text{bonus}(\text{path}(n)) \in [0, g(n)]$ (it can cover at most every move in the path), so the maximum possible additional underestimation introduced by the bonus term is bounded by $\lambda \cdot g(n)$ — i.e. the potential distortion grows

linearly with path depth, not with a fixed constant, which is a substantively different (and more concerning) failure mode than a bounded-error heuristic. This explains why, unlike classical bounded-suboptimal search (e.g. weighted A* with a fixed inflation factor and a provable suboptimality bound), we do not currently provide any formal bound on Trigger-Biased Search’s solution-length suboptimality; Section 6 instead reports this empirically (mean +7.4% moves at $\lambda = 1.0$). A natural mitigation is to cap bonus at a fixed maximum, $\text{bonus}_c(\text{path}(n)) = \min(\text{bonus}(\text{path}(n)), c)$ for some constant c , which bounds the maximum possible additional underestimation at a fixed $\lambda \cdot c$ regardless of path depth — we implemented and tested this rather than leaving it purely as a proposal.

Capped-bonus experiment. We re-ran Experiment 2’s exact $N = 60$ scramble set (seeds 2000–2059, same 40,000-node-per-depth budget) with bonus_c for $c \in \{1, 2, 4\}$ at $\lambda \in \{0.5, 1.0\}$, paired against the uncapped run reported above. Table 2 summarizes the outcome (Wilcoxon signed-rank on scrambles solved under both conditions). Two findings stand out. First, $c = 4$ is statistically

Table 2: Capped bonus vs. uncapped, $N = 60$, paired Wilcoxon signed-rank on scrambles solved by both, Holm–Bonferroni corrected across all $m = 10$ tests in this table

λ	c	solved	moves (p, r)	trig.ov. (p, r)
0.5	1	53/59	25.9/26.3 ($p=.16$)	.110/.186 ($p < 10^{-4}$, .57) [†]
0.5	2	53/59	25.8/26.3 ($p=.06$)	.124/.186 ($p < 10^{-3}$, .54) [†]
0.5	4	59/59	26.7/26.7 (0/59 differ)	.201/.201 (0/59 differ)
1.0	1	53/60	25.8/27.3 ($p=.003$, .41) [†]	.124/.248 ($p < 10^{-6}$, .70) [†]
1.0	2	58/60	26.7/27.5 ($p=.03$, .29)	.199/.253 ($p=.008$, .35) [†]
1.0	4	59/60	27.6/27.5 ($p=.18$; 2/59 differ)	.256/.256 ($p=1.0$; 3/59 differ)

solved = capped/uncapped, both out of 60; moves and trig.ov. cells show capped/uncapped means (paired subset). [†] significant at family-wise $\alpha = 0.05$ after Holm–Bonferroni correction across the 10 tests with a defined p -value in this table (2 cells at $\lambda=0.5, c=4$ have zero variance, excluded from the correction family). "X/Y differ" makes explicit that a non-significant or unlabelled cell can still contain genuine per-scramble differences that cancel in aggregate — only the $\lambda=0.5, c=4$ row is truly identical to uncapped.

indistinguishable from uncapped on every metric at both λ values in this move-length regime (F2L solves of roughly 20–33 moves), surviving Holm–Bonferroni correction for the full 10-test family: at $\lambda = 0.5$ this is a literal 0/59 difference, while at $\lambda = 1.0$ a handful of scrambles do differ (2–3/59) but average out to no detectable effect. Either way, the fixed bound $\lambda \cdot c$ is achieved *for free*, with no measurable cost to the trigger-overlap gains reported above. Second, and more surprising, tighter caps ($c = 1, 2$) do *not* cleanly trade trigger-overlap for fewer moves as we had expected — every trigger-overlap loss in Table 2 survives correction, but only the single most extreme move-count comparison ($\lambda = 1.0, c = 1$) does; the rest are not distinguishable from noise once corrected. Tighter caps instead measurably reduce the *solve success rate* (53–58/60 vs. 59–60/60 within the same node budget). Inspecting individual scrambles (e.g. seeds 2053, 2054,

2058) confirms the mechanism: for these scrambles the search times out under $c = 1$ or $c = 2$ but succeeds under $c = 4$ or uncapped, indicating the trigger bonus is not purely a cosmetic humanlikeness bias here — it also functions as informal *search-order guidance* that occasionally helps the bounded-node-budget search escape unproductive regions of the tree. Aggressively capping the bonus can therefore be counterproductive on both axes (humanlikeness *and* reliability) without reliably buying back move-count efficiency in return, which is why we report this as an empirical trade-off rather than recommend a single "safe" c a priori.

7.1 Pathological scrambles and search non-convergence

In the robustness-check run using $N = 100$ random-move scrambles (Section 7), 22% of scrambles (22/100) failed to complete within a 20-second wall-clock budget even at a reduced 40,000-node-per-depth search budget — notably, several such failures clustered consecutively (e.g. seeds 1059–1061 in our run), suggesting the underlying random-move scramble generator can produce runs of similarly structured, adversarial F2L configurations rather than i.i.d. difficulty. One specific seed (1013) was found to hang for over 5 minutes even at the reduced budget before we introduced a hard wall-clock timeout (SIGALRM-based) into our evaluation harness. We interpret this as evidence that the per-pair penalty-augmented A* heuristic used for F2L (Section 5) is occasionally led into pathologically unproductive search regions — an important robustness limitation distinct from, but potentially compounded by, the trigger-bias mechanism of Section 4. Note that both the primary run (random-state, Table 1) and this check used $\lambda = 0$ throughout (no trigger bias applied to the F2L stage); we separately measured the relationship between λ and convergence failure rate using the Experiment 2 data (Section 7), finding — counter to our initial expectation — that higher λ was associated with *fewer*, not more, convergence failures on that dataset.

8 Threats to Validity

- **Search non-convergence:** as reported in Section 7.1, 22% of scrambles in the random-move robustness-check run ($N = 100$) and 20% of scrambles in the primary random-state run ($N = 60$, Table 1) did not complete within a 20-second timeout in our evaluation hardware/budget setting; results are conditioned on the scrambles that did complete, which may not be a representative subsample (e.g. if difficulty correlates with trigger-overlap potential).
- **Construct validity of HLI:** the metric is hand-designed, validated against human judgment via Experiment 3 (Section 6.3). We additionally ran an adversarial sanity check: inserting move-neutral padding (a known trigger immediately followed by its own inverse, e.g. $RUR'U'$ then $URU'R'$, which cancels to the identity permutation — verified computationally)

into an otherwise unchanged solve inflates Trigger-overlap substantially (e.g. $0.333 \rightarrow 0.706$ after 10 inserted no-op pairs on one 63-move example), since neither Segmentability nor Pattern-conformity penalize non-productive move-count. This is a genuine limitation of Trigger-overlap as currently defined — it is not robust to adversarially padded input. It does *not*, however, compromise the results reported above: Trigger-Biased Search (Groups A/A+) is driven by A* minimizing true move-count $g(n)$ plus a heuristic bonus *capped* at λc (Section 7.1), giving it no incentive to insert cost-increasing padding for an unbounded, uncapped reward; Group B comes from an independent optimal solver; Group H is human-recorded. Future work applying HLI outside this move-minimizing setting (e.g. as a direct optimization target) should first add an explicit efficiency or move-count penalty to Trigger-overlap to close this gap.

- **Inter-rater reliability:** individual raters showed only modest agreement ($ICC(2,1) = 0.26$, below the commonly-used 0.4 acceptable-agreement threshold), but the $N = 28$ rater pool gives the mean rating used in the RQ-V1 correlation high reliability ($ICC(2,k) = 0.91$) — the RQ-V1 correlation should be read as a correlation with a stable group mean, not with any single rater’s judgment. *Note: these ICC figures were initially taken from the survey write-up; they have since been independently re-derived from the raw per-rater rating matrix (28 real CSV files, see `research/phase3_survey/compute_icc.py`), matching exactly: $ICC(2,1) = 0.2563$, $ICC(2,k) = 0.9061$.*
- **Corpus bias:** the finger-trick corpus T is manually curated (11 tricks), not derived from real solve logs; results may not generalize to solvers with different trigger vocabularies.
- **Scramble generation (checked):** Table 1 (Section 6) uses a WCA-style random-state scrambler (Section 5) as the primary result, since it more closely matches real competition scrambler methodology. We also re-ran Experiment 1 on $N = 100$ random-move scrambles (seeds 1000–1099, the generator used for Experiment 2) as a robustness check. Results were consistent between the two generators (78% vs. 80% solve rate; mean moves 67.2 vs. 66.3; mean HLI 0.655 vs. 0.658), suggesting our primary results are not an artifact of the specific scramble generator chosen. This random-move run is also where we observed the notable search non-convergence phenomenon (22% timeout, see Section 7.1); we did not observe a comparable failure cluster in the $N = 60$ random-state sample (lower, 20%, timeout rate), though the smaller sample size makes a direct comparison of failure clustering between the two generators inconclusive.
- **Single-domain generalizability:** results are specific to the Rubik’s Cube; claims about explicability more broadly should be limited accordingly.

9 Future Work

Independent replication of the exact-HLI recomputation. Section 6.3’s exact Segmentability/Pattern-conformity computation (`research/phase3_survey/recompute_hli_exact.py`) relies on a specific, hand-verified operationalization of “stage boundary” (first move index at which a postcondition is met, with strict ordering across stages — see script docstring for the reasoning behind this specific choice over alternatives we initially tried and rejected). An independent reimplementaion, or validation against manually-annotated ground-truth stage boundaries for a subset of stimuli, would strengthen confidence in this component of the pipeline.

Larger-scale replication. $N = 24$ stimuli is adequate for a pilot but modest for precise effect-size estimation; a follow-up with more stimuli per group (particularly more Group H solves from a wider pool of contributors, to reduce dependence on any single solver’s habits) would tighten confidence intervals on the group comparisons in Section 6.3.

Extending pattern-conformity to Cross/F2L. Pattern-conformity $P(\pi)$ currently applies only to OLL/PLL, since these are the only two stages with a finite, named case-to-algorithm table in our system; Cross and F2L are instead measured indirectly via Trigger-overlap rather than true Pattern-conformity. Building a canonical F2L case table (e.g. following the widely-used 41-case system in the speedcubing community) together with a case-recognition mechanism analogous to `verify_and_build_table()` (already used for PLL) would allow part of Trigger-overlap to be replaced by true Pattern-conformity for F2L, making HLI more accurately reflect this stage’s structure.

Alternative bias mechanisms. Trigger-Biased Search currently uses a single, hand-chosen hyperparameter λ . Follow-up work could compare this mechanism against alternatives: for instance, directly fine-tuning the pattern-database weights used for F2L rather than adding a separate bonus term, or learning the `bonus(path)` weighting from real human solve data (e.g. regressing on observed trigger frequency in CSTimer logs) instead of fixing λ by hand — turning the seed corpus T (Appendix A) from a manually curated list into a model learned directly from real solvers’ behavior.

10 Conclusion

Existing Rubik’s Cube solvers optimize for move-count, producing plans that are correct but uninterpretable to human learners; human speedcubers instead use CFOP, trading optimality for structure and learnability. We formalized this trade-off in the vocabulary of human-aware AI planning as a measurable *Human-Likeness Index* (HLI), and showed empirically ($N = 60$ random-state scrambles, robustness-checked on $N = 100$ random-move scrambles) that our

AI-CFOP solver scores substantially higher on this index than a computer-optimal Kociemba baseline (mean HLI 0.658 vs. 0 by construction), a gap that is robust across a wide range of component weightings (Section 4). We then introduced *Trigger-Biased A* Search*, a minimal, backward-compatible modification to the F2L heuristic that more than doubles trigger-overlap with human finger-trick patterns (from 9.0% to 25.5% at $\lambda = 1.0$) at a modest move-count cost (+7.4%), while — counter-intuitively — *improving* search convergence reliability on our test set, at the cost of increased worst-case compute time. Along the way, we uncovered and transparently reported a search non-convergence phenomenon (22% timeout rate) that constitutes a concrete robustness limitation of penalty-augmented heuristic search in this domain. Finally, a blinded human-judge study ($N = 24$ stimuli, 28 raters, Section 6.3) provided direct evidence for HLI’s construct validity ($\rho = 0.487$, $p = 0.016$) and, more strikingly, showed that Trigger-Biased Search moves AI-CFOP solves from statistically-distinguishable-from-human ($p = 0.005$) to statistically-indistinguishable-from-human ($p = 0.109$) in blinded human judgment — a concrete, human-validated demonstration that explicitly optimizing for human-recognizable patterns, rather than move-count alone, measurably changes how human observers perceive AI-generated solutions. We hope this work encourages further cross-pollination between classical combinatorial search AI and human-aware/explainable AI research in large, well-understood puzzle domains such as the Rubik’s Cube.

References

- [1] F. Agostinelli, S. McAleer, A. Shmakov, and P. Baldi. Solving the rubik’s cube with deep reinforcement learning and search. *Nature Machine Intelligence*, 1(8):356–363, 2019.
- [2] T. Chakraborti, A. Kulkarni, S. Sreedharan, D. E. Smith, and S. Kambhampati. Explicability? legibility? predictability? transparency? privacy? security? the emerging landscape of interpretable agent behavior. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, 2019.
- [3] P. Hingston. A turing test for computer game bots. *IEEE Transactions on Computational Intelligence and AI in Games*, 1(3):169–186, 2009.
- [4] H. Kociemba. Cube explorer / two-phase algorithm. <http://kociemba.org/cube.htm>, accessed July 26, 2026.
- [5] D. Wang (J Perm). Finger tricks. <https://jperm.net/3x3/fingertricks>, accessed August 17, 2026.
- [6] R. E. Korf. Finding optimal solutions to rubik’s cube using pattern databases. In *Proceedings of the National Conference on Artificial Intelligence (AAAI)*, 1997.

- [7] A. Kulkarni, Y. Zha, T. Chakraborti, S. G. Vadlamudi, Y. Zhang, and S. Kambhampati. Explicable robot planning as minimizing distance from expected behavior. *arXiv preprint arXiv:1611.05497*, 2016.
- [8] S. McAleer, F. Agostinelli, A. Shmakov, and P. Baldi. Solving the rubik’s cube without human knowledge. *arXiv preprint arXiv:1805.07470*, 2018.
- [9] T. Rokicki, H. Kociemba, M. Davidson, and J. Dethridge. The diameter of the rubik’s cube group is twenty. *SIAM Review*, 56(4):645–670, 2014.
- [10] M. Świechowski and D. Ślęzak. The many challenges of human-like agents in virtual game environments. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems (AA-MAS)*, pages 1996–2004. IFAAMAS, 2025. Also available as arXiv preprint arXiv:2505.20011.
- [11] K. Lakkaraju, T. Hassan, V. Khandelwal, P. Singh, C. Bradley, R. Shah, et al., and D. Wu. Allure: A multi-modal guided environment for helping children learn to solve a rubik’s cube with automatic solving and interactive explanations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 13185–13187, 2022.
- [12] K. Vanacore, J. Ocumpaugh, F. Agostinelli, D. Wu, S. Vuruma, and M. Irvin. Student engagement in ai-assisted complex problem solving: A pilot study of human-ai rubik’s cube collaboration. In *Proceedings of the IEEE International Conference on Healthcare Informatics (ICHI)*, pages 251–260, 2025. Also available as arXiv preprint arXiv:2511.01683.
- [13] Y. Zhang, S. Sreedharan, A. Kulkarni, T. Chakraborti, H. H. Zhuo, and S. Kambhampati. Plan explicability and predictability for robot task planning. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2017.

A Seed corpus T : 11 finger-tricks

Table 3 lists the full seed corpus T used to compute Trigger-overlap $O(\pi)$ (Section 4.1) and the bonus(path) term in Trigger-Biased Search (Section 4.1), taken directly from `research/finger_tricks.py`. These names and sequences are standard speedcubing-community terminology, not original to this work; see Wang [5] for a representative community reference. Each trigger is listed in its canonical (R-based) form; the system automatically generates the mirrored variant (L-based, swapping $R \leftrightarrow L$) when matching.

Table 3: Seed corpus T ($|T| = 11$)

Name	Move sequence
sexy_move	$RU R' U'$
sexy_move_inv	$U RU' R'$
reverse_sexy	$R' U' RU$
sledgehammer	$R' F R F'$
sledgehammer_inv	$F R' F' R$
hedgehog	$RU' R'$
triple_sexy	$(RU R' U')^3$
sune_trigger	$RU R' U RU^2 R'$
ubl_insert	$U' RU$
urf_insert	$U RU'$
niklas	$RU R' U' R' F R F'$